

РУКОВОДСТВО ПО РАЗВЁРТЫВАНИЮ И СОПРОВОЖДЕНИЮ

Установка, настройка и эксплуатация

Аудитория: специалисты подразделения информационных технологий, системные
администраторы

Версия документа: 3.1

Дата: 31.07.2026

СОДЕРЖАНИЕ

1. Общие сведения.....	5
1.1. Назначение документа.....	5
1.2. Оформление.....	5
2. Термины.....	5
3. Требования к оборудованию и программному обеспечению.....	6
3.1. Конфигурация сервера.....	6
3.2. Дисковое пространство.....	7
3.3. Программное обеспечение сервера.....	7
3.4. Требования к рабочим местам.....	8
3.5. Сетевые требования.....	8
4. Состав поставки.....	9
4.1. Что передаётся.....	9
4.2. Структура каталога системы.....	9
4.3. Принцип наложения файлов описания.....	10
4.4. Чего в поставке нет.....	11
5. Установка.....	11
5.1. Подготовка сервера.....	11
5.2. Установка Docker.....	12
5.3. Размещение поставки.....	13
5.4. Создание файла параметров.....	13
5.5. Выбор имени базы данных.....	14
5.6. Подготовка сертификата.....	14
5.7. Первый запуск.....	15
5.8. Создание учётной записи администратора.....	15
5.9. Проверка после установки.....	16
6. Параметры конфигурации.....	16
6.1. Как задаются параметры.....	16
6.2. Обязательные параметры.....	17
6.3. Основные параметры.....	18
6.4. Параметры базы данных.....	18
6.5. Пересчёт параметров под другую конфигурацию.....	19

7. Защищённое соединение и сертификат.....	19
7.1. Зачем это нужно.....	19
7.2. Выбор способа получения сертификата.....	20
7.3. Выпуск сертификата.....	20
7.4. Установка корневого сертификата на рабочие места.....	21
7.5. Продление сертификата.....	22
8. Проверочный стенд.....	23
8.1. Назначение.....	23
8.2. Отличия от рабочего сервера.....	24
8.3. Развёртывание стенда.....	24
9. Эксплуатация рабочего сервера.....	25
9.1. Основные команды.....	25
9.2. Повседневные операции.....	25
9.3. Управление учётными записями.....	26
9.4. Восстановление доступа супер-администратора.....	26
9.5. Режим технических работ.....	27
9.6. Переименование базы данных.....	28
9.7. Очистка накопленных данных.....	29
10. Порядок внесения изменений в систему.....	32
10.1. Общая схема.....	32
10.2. Получение исходного кода и внесение правки.....	32
10.3. Проверка изменения до выкладывания.....	33
10.4. Проверка на стенде.....	33
10.5. Выкладывание на рабочий сервер.....	34
10.6. Изменения схемы базы данных.....	35
10.7. Возврат к предыдущей версии.....	35
11. Резервное копирование и восстановление.....	36
11.1. Текущее состояние.....	36
11.2. Что подлежит копированию.....	36
11.3. Требования к процедуре.....	36
12. Мониторинг и журналы.....	37
12.1. Проверка состояния.....	37

12.2. Журналы приложения.....	37
12.3. Журналы внутри системы.....	38
12.4. Что контролировать.....	39
13. Диагностика и типовые неисправности.....	40
13.1. Порядок диагностики.....	40
13.2. Система недоступна, страница не открывается.....	40
13.3. Обратный прокси не запускается.....	41
13.4. Предупреждение о недоверенном сертификате.....	41
13.5. Прикладной сервер не запускается.....	42
13.6. Изменение параметра не подействовало.....	42
13.7. Заканчивается место на диске.....	42
13.8. Система работает медленно.....	43
13.9. Работник не видит раздел или получает отказ.....	43
13.10. Не приходят обновления без перезагрузки страницы.....	43
14. Перенос системы на другой сервер.....	44
14.1. Что переносится.....	44
14.2. Что меняется при переносе.....	44
14.3. Подготовка нового сервера.....	45
14.4. Выгрузка данных со старого сервера.....	45
14.5. Восстановление на новом сервере.....	47
14.6. Сертификат и запуск.....	48
14.7. Проверка после переноса.....	48
14.8. Переключение и вывод прежнего сервера из работы.....	49
Приложение А. Порты и сетевые доступы.....	49
Приложение Б. Перечень параметров конфигурации.....	50
Приложение В. Файлы, необходимые для запуска.....	52
Приложение Г. Проверочный лист ввода в эксплуатацию.....	53

1. Общие сведения

1.1. Назначение документа

Документ содержит порядок развёртывания системы электронной подачи заявок «Бюро пропусков», её настройки, ввода в эксплуатацию и последующего сопровождения.

Документ рассчитан на специалиста, не работавшего ранее с данной системой. Порядок действий описан от состояния «есть сервер с установленной операционной системой» до состояния «система принимает заявки». Команды приведены в виде, пригодном для непосредственного выполнения, с указанием ожидаемого результата каждого шага.

Назначение системы, устройство и состав возможностей описаны в отдельном документе, «Техническое описание системы».

1.2. Оформление

Команды, имена файлов и параметры набраны моноширинным шрифтом. Угловые скобки означают подстановку: <домен> заменяется фактическим значением без скобок.

Сведения, требующие повышенного внимания, выделяются:

ПРИМЕЧАНИЕ. Пояснение, полезное для понимания, но не влияющее на порядок действий.

ВАЖНО. Пропуск приведёт к неверной работе или к дополнительной работе по исправлению.

ОПАСНО. Последствия необратимы. Утрата данных или утрата доступа к ним.

2. Термины

Таблица 2.1 — Термины, применяемые в документе

Термин	Значение
Рабочий сервер	Сервер, на котором система работает для сотрудников предприятия. В обиходе «прод», «продакшн»
Проверочный стенд	Отдельный сервер с той же системой для проверки изменений до выкладывания на рабочий. В обиходе «стенд», «стейдж»

Продолжение таблицы 2.1

Термин	Значение
Контейнер	Изолированная среда, в которой запускается одна часть системы. Части не мешают друг другу и не требуют установки в операционную систему
Образ	Заготовка, из которой запускается контейнер. Собирается из исходного кода один раз, затем запускается сколько угодно раз
Том	Хранилище данных контейнера, живущее отдельно от него. Пересоздание контейнера данные не затрагивает
Обратный прокси	Часть системы, принимающая соединения снаружи и распределяющая их между остальными частями
Сертификат	Электронный документ, которым сервер подтверждает браузеру свою подлинность и обеспечивает защищённое соединение
Удостоверяющий центр	Сторона, подписывающая сертификаты. Браузер доверяет сертификату, если доверяет подписавшему его центру
Файл параметров	Файл .env в каталоге системы, содержащий пароли, ключи и настройки конкретной установки

3. Требования к оборудованию и программному обеспечению

3.1. Конфигурация сервера

Конфигурация рассчитана на устойчивую работу при 1000-1500 одновременно работающих пользователях. Параметры базы данных и прикладного сервера, поставляемые в комплекте, подобраны именно под неё. Значение получено расчётом; нагрузочные испытания на целевом оборудовании подтвердят или уточнят его.

Таблица 3.1 — Рекомендуемая конфигурация сервера

Параметр	Значение
Процессор	6 физических ядер с частотой от 3,4 ГГц, например Intel Xeon E-2236
Оперативная память	32 ГБ с коррекцией ошибок
Дисковая подсистема	Два твердотельных накопителя от 480 ГБ в зеркальном массиве
Сеть	Один адрес, пропускная способность от 100 Мбит/с

ВАЖНО. Зеркальный массив обязателен. В базе хранятся персональные данные, и при отказе одиночного накопителя восстановление возможно только из резервной копии, то есть с потерей всех изменений, сделанных после её создания.

Минимальная конфигурация для пробной эксплуатации и проверочного стенда: 4 ядра, 8 ГБ памяти, 100 ГБ дискового пространства. При такой конфигурации параметры базы данных из комплекта поставки требуется уменьшить, иначе службе не хватит памяти. Порядок изменения описан в подразделе 6.5.

3.2. Дисковое пространство

Таблица 3.2 — Ориентировочное потребление дискового пространства

Потребитель	Оценка
Образы контейнеров и операционная система	15-20 ГБ
База данных без журналов	1-3 ГБ за год работы
Журнал запросов	Основной потребитель. При высокой активности подробные записи за 30 суток занимают десятки гигабайт
Загруженные файлы	Фотографии, шаблоны бланков, приложения к заявкам
Резервные копии	Не менее трёх объёмов базы данных

Журнал запросов растёт быстрее остального, поэтому глубина хранения подробных записей вынесена в параметр `REQUEST_LOG_DETAIL_DAYS`. Уменьшение до 7-14 суток кратно снижает занимаемый объём и на работу системы не влияет.

Служебные записи, обесценивающиеся сами, система удаляет ежесуточно без участия человека. Историю, слепки таблиц и суточные итоги журнала при нехватке места удаляют командой из подраздела 9.7.

3.3. Программное обеспечение сервера

Таблица 3.3 — Требования к программному обеспечению

Компонент	Требование
Операционная система	Любой дистрибутив Linux с поддержкой контейнеров: Ubuntu Server 22.04 и новее, Debian 12 и новее, RHEL-совместимые (Rocky, Alma), Astra Linux, РЕД ОС
Docker Engine	Версия 25 и новее
Docker Compose	Версия 2.24 и новее, вызывается как <code>docker compose</code>
openssl	Генерация секретов и сертификата
certbot	Только при выпуске сертификата через общедоступный удостоверяющий центр

Отдельно устанавливать Go, Node.js, PostgreSQL или nginx не требуется: всё необходимое собирается и работает внутри контейнеров.

3.4. Требования к рабочим местам

Браузер актуальной версии: Chrome, Yandex Browser, Firefox, Edge.
Разрешение экрана от 1280 точек по ширине для рабочих мест бюро и постов. Интерфейс приспособлен и к мобильным устройствам.

Дополнительная установка чего-либо на рабочие места не требуется, за одним исключением: если сертификат сервера выпущен собственным удостоверяющим центром организации, потребуется однократно установить его корневой сертификат. Подробнее в подразделе 7.4.

3.5. Сетевые требования

Наружу открываются только два порта. Остальные порты частей системы существуют исключительно во внутренней сети контейнеров и не должны быть доступны с рабочих мест.

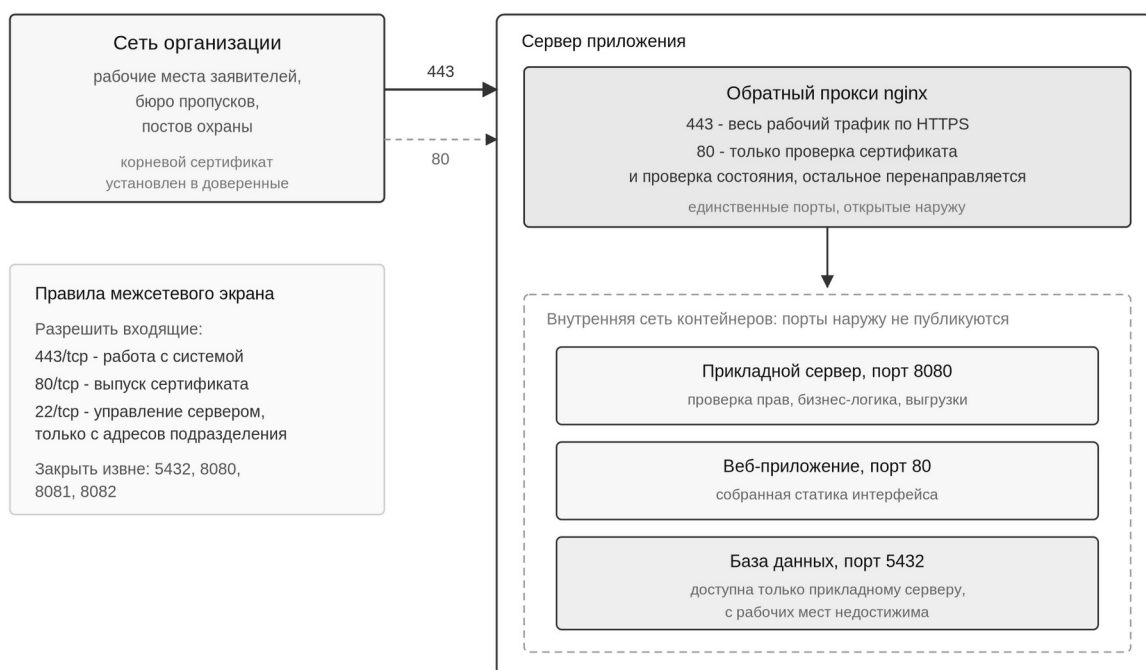


Рисунок 3.1 — Разграничение сетевых доступов

Полный перечень портов с указанием области доступности приведён в приложении А.

4. Состав поставки

4.1. Что передаётся

Поставка представляет собой архив с исходным кодом системы и всеми файлами, необходимыми для её сборки и запуска. Готовые собранные образы не передаются: они собираются на сервере при установке, что исключает несовместимость с конкретной операционной системой.

Таблица 4.1 — Состав поставки

Элемент	Назначение
Исходный код серверной части и интерфейса	Из него на сервере собираются образы контейнеров
Файлы описания контейнеров	Определяют состав частей системы, их связи, ограничения ресурсов
Конфигурация обратного прокси	Правила распределения запросов, защищённое соединение, заголовки безопасности
Сборочный файл	Набор коротких команд управления вместо длинных команд Docker
Сценарий подготовки параметров	Создаёт файл параметров и генерирует пароли и ключи
Сценарий подготовки сертификата	Выпуск и продление сертификата, три способа на выбор
Эксплуатационная документация	Настоящий документ, техническое описание и руководства пользователей

Перечень конкретных файлов, без которых система не запустится, приведён в приложении В.

4.2. Структура каталога системы

После распаковки поставки в рабочем каталоге образуется следующая структура. Знать её требуется для изменения настроек и для диагностики.

Листинг 4.1 — Структура каталога системы

/opt/systemburo/	
— docker-compose.base.yml	состав служб, общая часть
— docker-compose.prod.yml	наложение: рабочий сервер
— docker-compose.staging.yml	наложение: стенд
— docker-compose.yml	локальная разработка
— Dockerfile	образ прикладного сервера
— Makefile	команды управления
— .env	параметры и секреты
— cmd/ internal/ docs/	код серверной части
— frontend/	
— Dockerfile	образ веб-приложения
— nginx.conf	отдача статики в образе
— src/ package.json	код интерфейса
— nginx/	
— nginx.conf	прокси рабочего сервера
— staging.conf	прокси стенда
— Dockerfile.staging	образ прокси для стенда
— certs/	сертификат и ключ
— acme-webroot/	выпуск сертификата
— scripts/	
— init-env.sh	создание файла параметров
— init-tls.sh	выпуск и продление сертификата
— maintenance-off.sh	аварийное снятие режима технических работ

4.3. Принцип наложения файлов описания

Ни рабочий сервер, ни стенд не имеют самостоятельного файла описания. Состав служб задан один раз в базовом файле, а различия среды добавляются наложением поверх него.

Листинг 4.2 — Порядок указания файлов описания

```
docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
ps
```

Именно поэтому во всех командах настоящего документа указываются два файла подряд. Порядок существенен: базовый идёт первым, наложение вторым, значения из второго заменяют значения из первого.

ВАЖНО. Команда, в которой указан только файл наложения, работать не будет. Служба базы данных определена в базовом файле, и без него Docker сообщит, что такой службы не существует. Это частая причина ошибок при копировании команд из посторонних источников.

Сокращённые команды из сборочного файла (make deploy-up, make deploy-logs и прочие) подставляют оба файла самостоятельно, поэтому при их использовании об этом думать не требуется.

4.4. Чего в поставке нет

В поставку намеренно не входят пароли, ключи шифрования и сертификаты. Они создаются на сервере при установке и остаются только у организации, где развёрнута система. Это означает, что разработчик не имеет доступа к данным работающей системы.

Не входят и данные: справочники, учётные записи, заявки. Система разворачивается пустой, после чего заполняется силами бюро пропусков через интерфейс.

5. Установка

Раздел описывает установку с нуля на сервер с установленной операционной системой. Результат: работающая система, доступная по защищённому соединению, с заведённой учётной записью администратора.

Команды выполняются от имени пользователя, входящего в группу `docker`.

5.1. Подготовка сервера

Установите служебные утилиты.

Листинг 5.1 — Установка служебных пакетов

```
# Ubuntu, Debian, Astra Linux
sudo apt update && sudo apt install -y openssl curl ca-certificates

# RHEL-совместимые, РЕД ОС
sudo dnf install -y openssl curl ca-certificates
```

Создайте отдельного пользователя для работы с системой. Работать под учётной записью суперпользователя не следует.

Листинг 5.2 — Создание пользователя

```
sudo useradd -m -s /bin/bash buro
sudo usermod -aG docker buro
```

Задайте часовой пояс: от него зависят отметки времени в журналах.

```
sudo timedatectl set-timezone Europe/Moscow
```

5.2. Установка Docker

Версия из штатных репозиториев дистрибутива нередко устарела и может не поддерживать применяемый формат файлов описания. Устанавливайте из репозитория разработчика.

Листинг 5.4 — Установка Docker

```
# Ubuntu, Debian
curl -fsSL https://get.docker.com | sudo sh

# RHEL-совместимые
sudo dnf config-manager --add-repo \
  https://download.docker.com/linux/centos/docker-ce.repo
sudo dnf install -y docker-ce docker-ce-cli containerd.io \
  docker-compose-plugin
sudo systemctl enable --now docker
```

Для Astra Linux и РЕД ОС используйте пакеты из репозитория дистрибутива, если политика организации запрещает подключение внешних репозиториев.

Проверьте установку.

Листинг 5.5 — Проверка установки Docker

```
docker --version
docker compose version
```

Ожидаемый результат: обе команды выводят номер версии, у Docker Compose не ниже 2.24. Вторая обязана вызываться именно как `docker compose`, через пробел. Если она сообщает об ошибке, а работает `docker-compose` через дефис, установлена устаревшая версия, и файлы описания системы обработаны не будут.

ВАЖНО. Версия Docker Compose ниже 2.24 не понимает применяемый в описании рабочего сервера синтаксис снятия ограничений памяти и прервёт запуск сообщением о неверном формате файла. Версия из штатного репозитория дистрибутива нередко оказывается старше.

5.3. Размещение поставки

Листинг 5.6 — Распаковка поставки

```
sudo mkdir -p /opt/systemburo
sudo chown buro:buro /opt/systemburo
tar -xzf systemburo-<версия>.tar.gz -C /opt/systemburo --strip-components=1
cd /opt/systemburo
```

Все дальнейшие команды выполняются из этого каталога.

ВАЖНО. Имя каталога определяет имена томов с данными. Если каталог впоследствии переименовать, Docker посчитает установку новой и создаст пустые тома, а система запустится с пустой базой. Данные при этом не теряются, но чтобы полностью исключить такую ситуацию, добавьте в файл параметров строку `COMPOSE_PROJECT_NAME=systemburo` сразу после его создания. Тогда имена томов перестанут зависеть от имени каталога.

5.4. Создание файла параметров

Файл параметров создаётся сценарием, который сам генерирует стойкие случайные пароли и ключи. Задавать их вручную не следует.

Листинг 5.7 — Создание файла параметров

```
make init-production DOMAIN=<адрес системы>
```

Вместо `<адрес системы>` подставьте доменное имя, по которому система будет доступна, либо её сетевой адрес.

Ожидаемый результат: создан файл `.env`, на экран выведены сгенерированные учётные данные.

ВАЖНО. Перенесите выведенные значения в хранилище секретов организации. Повторно сценарий их не покажет, но все они остаются в файле `.env`. Из этого следует обратное требование: файл `.env` содержит пароли и ключ шифрования открытым текстом, его нельзя копировать в общедоступные каталоги, прикладывать к обращениям и включать в резервные копии общего доступа.

Ограничьте доступ к файлу параметров.

Листинг 5.8 — Ограничение доступа к файлу параметров

```
chmod 600 .env
```

Особое внимание к параметру DATA_ENCRYPTION_KEY. Им зашифрованы сведения документов, удостоверяющих личность.

ОПАСНО. При утрате ключа шифрования расшифровать персональные данные невозможно. Резервная копия базы не поможет: она содержит те же зашифрованные значения. Ключ должен храниться отдельно от сервера и отдельно от резервных копий.

Сценарий намеренно отказывается перезаписывать существующий файл параметров.

ОПАСНО. Не запускайте создание параметров повторно на работающей системе. Будет сгенерирован новый ключ шифрования, и ранее зашифрованные персональные данные перестанут читаться.

5.5. Выбор имени базы данных

Имя базы задаётся параметром DB_NAME в файле параметров. Если требуется имя, отличное от заданного по умолчанию, измените его **до первого запуска**: тогда база сразу создастся с нужным именем.

Листинг 5.9 — Пример изменения имени базы до первого запуска

```
DB_NAME=buropropuskov  
DATABASE_URL=postgres://postgres:<пароль>@db/buropropuskov
```

Переименование уже работающей базы описано в подразделе 9.6.

5.6. Подготовка сертификата

Сертификат готовится до первого запуска: обратный прокси не стартует, если файла сертификата нет. Способы выпуска и выбор между ними описаны в разделе 7. Для установки внутри сети организации достаточно выполнить:

Листинг 5.10 — Выпуск сертификата для внутренней сети

```
./scripts/init-tls.sh self <адрес системы> <IP-адрес сервера>
```

Ожидаемый результат: на экран выведены сведения о выпущенном сертификате, включая срок действия, и указан путь к корневому сертификату для установки на рабочие места.

5.7. Первый запуск

Сборка образов выполняется на сервере и занимает от нескольких минут до получаса в зависимости от производительности.

Листинг 5.11 — Сборка и запуск

```
make deploy-build  
make deploy-up
```

Первый запуск дольше последующих: создаётся схема базы данных, устанавливаются расширения, строятся индексы.

Листинг 5.12 — Наблюдение за запуском

```
make deploy-logs
```

Ожидаемый результат: в журнале нет сообщений уровня `error`, последние строки сообщают о запуске сервера.

5.8. Создание учётной записи администратора

Листинг 5.13 — Создание администратора

```
make deploy-seed PASS=<пароль администратора>
```

Команда создаёт учётную запись `buropropuskov` с правами супер-администратора и заводит для неё организацию и компанию «Бюро пропусков».

Обязательные справочники - типы работников и разделы встроенного руководства - заполняются раньше, при первом запуске прикладного сервера. Поэтому команда выполняется только после успешного запуска: без справочника типов она завершится ошибкой.

ВАЖНО. Аргумент `PASS` обязателен. Без него будет установлен пароль по умолчанию, известный из состава поставки.

Пароль хранится в виде необратимой свёртки и в открытом виде нигде не сохраняется.

Демонстрационное наполнение (make deploy-seed-demo) на рабочем сервере запускать не следует: оно создаёт вымышленные заявки, организации и транспорт, которые придётся удалять вручную.

5.9. Проверка после установки

Выполните проверки последовательно. Каждая проверяет свой уровень, и успех предыдущей не гарантирует успех следующей.

Таблица 5.1 — Проверки после установки

Проверка	Команда или действие	Ожидаемый результат
Части системы запущены	<code>docker compose -f docker-compose.base.yml -f docker-compose.prod.yml ps</code>	Четыре службы в состоянии Up, база в состоянии healthy
Сервер отвечает	<code>curl -s http://localhost/health</code>	<code>{"status":"ok"}</code>
Защищённое соединение работает	<code>curl -sI https://<адрес>/</code>	Код 200, присутствует заголовок Strict-Transport-Security
Незащищённое перенаправляется	<code>curl -sI http://<адрес>/</code>	Код 301, заголовок Location начинается с https://
Вход выполняется	Открыть систему в браузере, войти под администратором	Открывается главный экран
Права применяются	Открыть раздел администрирования	Разделы управления доступны

ПРИМЕЧАНИЕ. Проверка /health подтверждает только то, что процесс жив. К базе данных она не обращается, поэтому успешный ответ при недоступной базе возможен. Полноценной проверкой является вход в систему.

6. Параметры конфигурации

6.1. Как задаются параметры

Параметры передаются прикладному серверу переменными окружения. Файл `.env` в каталоге системы подставляет значения в описание контейнеров.

ВАЖНО. Описание рабочего сервера передаёт в контейнер не весь файл параметров, а только явно перечисленный набор. Параметры вне этого набора в контейнер не попадают и работают на встроенных значениях, даже если прописаны в `.env`.

Через файл параметров действуют: DATABASE_URL, JWT_SECRET, JWT_REFRESH_SECRET, JWT_ACCESS_TTL, JWT_REFRESH_TTL, LOG_LEVEL, DATA_ENCRYPTION_KEY, CORS_ALLOWED_ORIGINS, SWAGGER_ENABLED, а также DB_USER, DB_PASSWORD, DB_NAME, из которых собирается строка подключения.

Чтобы изменить любой другой параметр, добавьте его в раздел `environment` службы `backend` файла `docker-compose.prod.yml` и перезапустите систему.

Листинг 6.1 — Изменение параметра, не передаваемого через файл параметров

```
backend:
  environment:
    RESET_TIMEZONE: "Asia/Yekaterinburg"
    REQUEST_LOG_DETAIL_DAYS: "14"
```

Листинг 6.2 — Применение изменений

```
make deploy-up
```

Ожидаемый результат: контейнер прикладного сервера пересоздан, новые значения действуют. Проверить можно командой из подраздела 13.6.

6.2. Обязательные параметры

Три параметра обязательны. При их отсутствии или недопустимом значении прикладной сервер завершает работу с ошибкой, а не запускается в ослабленном режиме.

Таблица 6.1 — Обязательные параметры

Параметр	Проверка при запуске
DATABASE_URL	Должен начинаться с <code>postgres://</code> или <code>postgresql://</code>
JWT_SECRET	Не короче 32 символов
JWT_REFRESH_SECRET	Не короче 32 символов

Раздельные ключи для маркера доступа и маркера продления принципиальны: при совпадении маркер продления можно было бы предъявить вместо маркера доступа и продлить сеанс бесконечно. Сценарий подготовки параметров генерирует их независимо, поэтому при штатной установке они заведомо различны.

ВАЖНО. Совпадение этих двух ключей система при запуске не проверяет. Задавая их вручную, убедитесь, что они разные.

6.3. Основные параметры

Полный перечень приведён в приложении Б. Здесь перечислены параметры, которые чаще всего требуется изменять.

Таблица 6.2 — Часто изменяемые параметры

Параметр	По умолчанию	Назначение
LOG_LEVEL	info	Подробность журнала: debug, info, warn, error
RESET_TIMEZONE	Europe/Moscow	Часовой пояс суточного сброса территориальных статусов
REQUEST_LOG_DETAIL_DAYS	30	Глубина хранения подробных записей журнала запросов
RATE_LIMIT_PER_MINUTE	200	Допустимое число запросов в минуту от одного работника
UPLOAD_MAX_FILE_SIZE	10485760	Наибольший размер загружаемого файла, 10 МБ
SWAGGER_ENABLED	false	Публикация описания программного интерфейса
REQUIRE_ENCRYPTION	false	Запрет запуска без ключа шифрования персональных данных

ВАЖНО. На рабочем сервере установите REQUIRE_ENCRYPTION в значение true. Это исключает ситуацию, когда система запустилась без ключа шифрования и записала персональные данные в открытом виде.

6.4. Параметры базы данных

Заданы в файле описания рабочего сервера и подобраны под конфигурацию из подраздела 3.1.

Таблица 6.3 — Параметры базы данных и среды выполнения

Параметр	Значение	Смысл
shared_buffers	8GB	Память под собственный кэш страниц базы
effective_cache_size	20GB	Оценка доступной памяти под кэш, влияет на выбор плана запроса
work_mem	32MB	Память на одну операцию сортировки или соединения
maintenance_work_mem	1GB	Память под построение индексов и обслуживание

Продолжение таблицы 6.3

Параметр	Значение	Смысл
max_connections	150	Наибольшее число одновременных подключений
max_wal_size	4GB	Объём журнала предзаписи между контрольными точками
random_page_cost	1.1	Стоимость произвольного чтения, значение для твердотельных накопителей
effective_io_concurrency	200	Число параллельных операций ввода-вывода
GOMAXPROCS	6	Число ядер, используемых прикладным сервером
GOMEMLIMIT	6GiB	Порог, при котором усиливается освобождение памяти

6.5. Пересчёт параметров под другую конфигурацию

При установке на сервер с иными характеристиками параметры базы данных требуется пересчитать, иначе службе не хватит памяти либо она не использует доступную.

Таблица 6.4 — Правила пересчёта

Параметр	Правило
shared_buffers	Около четверти оперативной памяти
effective_cache_size	Около двух третей оперативной памяти
maintenance_work_mem	Около 1/32 оперативной памяти
GOMAXPROCS	Равно числу ядер процессора
GOMEMLIMIT	Около одной пятой оперативной памяти

Пример для сервера с 8 ГБ памяти и 4 ядрами: shared_buffers=2GB, effective_cache_size=5GB, maintenance_work_mem=256MB, GOMAXPROCS=4, GOMEMLIMIT=1600MiB.

7. Защищённое соединение и сертификат

7.1. Зачем это нужно

Система работает только по защищённому соединению. Причина не формальная: через неё передаются пароли и персональные данные, которые по незащищённому соединению видны любому, кто имеет доступ к сети между рабочим местом и сервером.

Кроме того, продление сеанса технически работает только по защищённому соединению: браузеру запрещено сохранять маркер продления, полученный по обычному протоколу. Поэтому без сертификата вход в систему не заработает вовсе.

ПРИМЕЧАНИЕ. Сертификат нужен серверу, а не пользователям. Сотрудникам выдаются обычные имя и пароль. Никаких персональных сертификатов сотням работников выдавать не требуется.

7.2. Выбор способа получения сертификата

Сертификат должен быть подписан стороной, которой доверяет браузер. Отсюда три возможных способа, различающихся тем, кто подписывает.

Таблица 7.1 — Способы получения сертификата

Способ	Когда подходит	Что потребуется на рабочих местах
Собственный удостоверяющий центр	Система работает внутри сети, внешнего доменного имени нет	Однократная установка одного корневого сертификата, централизованно групповой политикой
Общедоступный удостоверяющий центр	У организации есть доменное имя, которым она владеет	Ничего, доверие настроено в браузерах изначально
Удостоверяющий центр организации	В организации развёрнута собственная служба сертификатов	Ничего, корневой сертификат уже установлен на машинах домена

ПРИМЕЧАНИЕ. Второй способ часто оказывается удобнее первого, и он применим, даже когда сервер стоит внутри сети и недоступен извне. Владение доменом подтверждается записью в системе доменных имён, а не доступом к серверу. Если у организации есть свой домен, этот путь избавляет от установки чего-либо на рабочие места.

7.3. Выпуск сертификата

Собственный удостоверяющий центр. Применяется при работе внутри сети организации.

Листинг 7.1 — Выпуск сертификата собственным центром

```
./scripts/init-tls.sh self <адрес системы> <IP-адрес сервера>
```

Сценарий создаёт корневой сертификат удостоверяющего центра и подписанный им сертификат сервера. Указание сетевого адреса вторым аргументом позволяет обращаться к системе и по имени, и по адресу.

Ожидаемый результат: выведены сведения о сертификате и путь к корневому сертификату `nginx/certs/ca/ca.crt`.

Общедоступный удостоверяющий центр. Применяется при наличии доменного имени, доступного из сети общего пользования, и открытого порта 80.

Листинг 7.2 — Выпуск сертификата общедоступным центром

```
./scripts/init-tls.sh self <домен>  
make deploy-up  
./scripts/init-tls.sh acme <домен> <адрес электронной почты>
```

Порядок двухшаговый: сначала создаётся временный сертификат, чтобы прокси смог запуститься, затем выпускается настоящий. Без первого шага прокси не стартует, а без работающего прокси невозможно подтвердить владение доменом.

Готовый сертификат. Применяется, если сертификат приобретён либо выпущен удостоверяющим центром организации.

Листинг 7.3 — Установка готового сертификата

```
./scripts/init-tls.sh import <файл цепочки> <файл ключа>
```

Сценарий сверяет соответствие ключа сертификату и отказывается устанавливать несовпадающую пару. Это защищает от типичной ошибки, при которой прокси перестаёт запускаться уже после перезапуска, когда разбираться некогда.

7.4. Установка корневого сертификата на рабочие места

Шаг обязателен **только** при выпуске сертификата собственным удостоверяющим центром.

ВАЖНО. Без этого шага работники увидят предупреждение о недоверенном сертификате и не смогут его пропустить. Кнопки «перейти всё равно» не будет: система требует обязательного защищённого соединения, а это запрещает браузеру такой обход. Установка корневого сертификата снимает предупреждение полностью.

Файл для установки: `nginx/certs/ca/ca.crt`. Он одинаков для всех рабочих мест.

Централизованно в домене. Групповая политика: «Конфигурация компьютера» → «Политики» → «Конфигурация Windows» → «Параметры безопасности» → «Политики открытого ключа» → «Доверенные корневые центры сертификации», импорт файла. Политика применится на рабочих местах при очередном обновлении.

Отдельная машина Windows. Из командной строки с правами администратора:

Листинг 7.4 — Установка корневого сертификата в Windows

```
certutil -addstore -f Root ca.crt
```

Рабочее место Linux.

Листинг 7.5 — Установка корневого сертификата в Linux

```
sudo cp ca.crt /usr/local/share/ca-certificates/buro-ca.crt  
sudo update-ca-certificates
```

ОПАСНО. Приватный ключ удостоверяющего центра `nginx/certs/ca/ca.key` позволяет выпустить сертификат на любое имя, которому будут доверять все рабочие места организации. Он не должен покидать сервер и попадать в резервные копии общего доступа.

7.5. Продление сертификата

Сертификат, выпущенный собственным центром, действует 825 суток. За месяц до истечения выпустите новый той же командой: корневой сертификат переиспользуется, поэтому повторно устанавливать его на рабочие места не потребуется.

Листинг 7.6 — Продление сертификата собственного центра

```
./scripts/init-tls.sh self <адрес системы> <IP-адрес сервера>  
docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \  
exec nginx nginx -s reload
```

Сертификат общедоступного центра действует 90 суток и требует регулярного продления. Настройте задание в планировщике.

Листинг 7.7 — Задание на автоматическое продление

```
30 3 * * * cd /opt/systemburo && ./scripts/init-tls.sh renew \  
>> /var/log/systemburo-tls.log 2>&1
```

Команда безопасна для ежедневного запуска: она обращается за новым сертификатом только когда до истечения остаётся менее 30 суток, и перезагружает прокси лишь при фактическом обновлении файла.

Листинг 7.8 — Проверка срока действия

```
openssl x509 -in nginx/certs/fullchain.pem -noout -dates
```

Ожидаемый результат: две строки с датами начала и окончания действия.

8. Проверочный стенд

8.1. Назначение

Проверочный стенд представляет собой второй экземпляр системы, развёрнутый отдельно от рабочего сервера. Он нужен, чтобы проверять изменения и настройки до их применения на рабочем сервере, обучать сотрудников без риска для данных и воспроизводить обращения пользователей.

ВАЖНО. Не разворачивайте стенд на том же сервере, где работает рабочий экземпляр. Они разделят память и дисковую подсистему, и проверка нагрузки на стенде скажется на работе сотрудников.

8.2. Отличия от рабочего сервера

Таблица 8.1 — Отличия стенда от рабочего сервера

Свойство	Стенд	Рабочий сервер
Доступ	Закрыт запросом имени и пароля на входе	Открыт, разграничение внутри системы
Подробность журнала	debug	info
Панель управления базой данных	Разворачивается, доступна по адресу /pgadmin/	Не разворачивается
Наполнение	Допустимы демонстрационные данные	Только рабочие данные

ПРИМЕЧАНИЕ. Запрос имени и пароля на входе в стенд решает одну задачу, не пустить посторонних. Он не заменяет разграничение прав внутри системы.

8.3. Развёртывание стенда

Порядок совпадает с рабочим сервером, отличаются вызываемые команды.

Листинг 8.1 — Развёртывание стенда

```
make init-staging DOMAIN=<адрес стенда>
make staging-build
make staging-up
make staging-seed PASS=<пароль администратора>
```

Ожидаемый результат: стенд доступен по указанному адресу, при открытии запрашивает имя и пароль, выведенные сценарием подготовки параметров.

ПРИМЕЧАНИЕ. Обратный прокси стенда принимает соединения по незащищённому протоколу и сертификата не требует - выпускать его командой из раздела 7 не нужно. Если стенд должен быть доступен по защищённому соединению, оно обеспечивается внешним прокси перед ним.

Демонстрационное наполнение добавляется отдельно:

Листинг 8.2 — Добавление демонстрационных данных

```
make staging-seed-demo PASS=<пароль администратора>
```


9. Эксплуатация рабочего сервера

9.1. Основные команды

Таблица 9.1 — Команды управления системой

Действие	Команда
Запуск	make deploy-up
Остановка	make deploy-down
Пересборка образов	make deploy-build
Просмотр журналов	make deploy-logs
Состояние служб	docker compose -f docker-compose.base.yml -f docker-compose.prod.yml ps
Обзор занятого места	make deploy-storage
Очистка накопленных данных	make deploy-cleanup

Остановка прерывает работу пользователей. Прикладной сервер завершается корректно: перестаёт принимать новые запросы, даёт до 10 секунд на завершение текущих и только затем останавливается, поэтому незавершённых операций записи при штатной остановке не остаётся.

9.2. Повседневные операции

Таблица 9.2 — Регламент повседневных работ

Операция	Периодичность	Кто выполняет
Контроль состояния служб	Ежедневно	Подразделение ИТ
Просмотр журнала на ошибки	Ежедневно	Подразделение ИТ
Просмотр журнала отказов в доступе	Еженедельно	Подразделение ИТ либо оператор системы с правом просмотра
Контроль свободного места	Еженедельно	Подразделение ИТ
Контроль срока действия сертификата	Ежемесячно	Подразделение ИТ
Проверка восстановления из резервной копии	Ежемесячно	Подразделение ИТ

ПРИМЕЧАНИЕ. Наблюдение за журналами системы и разбор обращений пользователей доступны не только подразделению ИТ, но и операторам бюро пропусков, если им выдано соответствующее право. Журналы открываются прямо в интерфейсе системы, доступ к серверу для этого не нужен.

9.3. Управление учётными записями

Заведение и блокировка работников выполняются в интерфейсе системы и описаны в руководстве администратора. С точки зрения эксплуатации существенны три обстоятельства.

Блокировка действует немедленно: заблокированный работник теряет все права, а его сеансы прекращаются, поэтому продолжить работу в уже открытой вкладке он не сможет.

Блокировку входа после неудачных попыток пароля не следует путать с блокировкой работника: первая временная и снимается сама по истечении срока. Срок растёт с каждой новой серией из пяти неудач - минута, пять, пятнадцать, тридцать минут, час, - поэтому работник, который перебирает забытый пароль, ждёт всё дольше. Чтобы не заставлять его ждать, администратор снимает такую блокировку кнопкой в карточке работника; обращение к серверу для этого не нужно.

Изменение прав вступает в силу с задержкой до 30 секунд, поскольку результат вычисления прав кэшируется. Для открытых страниц с автообновлением задержка может достигать 10 минут.

9.4. Восстановление доступа супер-администратора

Супер-администратор в системе один. При утрате доступа к нему восстановление выполняется повторным запуском команды создания учётной записи: она не создаёт вторую запись, а обновляет пароль существующей.

Листинг 9.1 — Смена пароля супер-администратора

```
make deploy-seed PASS=<новый пароль>
```

Ожидаемый результат: команда завершается без ошибок, вход выполняется с новым паролем. Прочие данные не затрагиваются.

9.5. Режим технических работ

Режим включается супер-администратором в разделе системного управления. При включённом режиме все работники, кроме супер-администратора, попадают на страницу технических работ и не могут выполнять действия.

Режим следует включать перед обновлением и любыми работами, затрагивающими целостность данных. Он не останавливает фоновые задачи и не прерывает уже выполняющиеся запросы.

При включении заполняются четыре поля. Все они видны пользователю на странице технических работ, поэтому заполнять их следует так, как если бы вы отвечали на вопрос «когда всё заработает и к кому обращаться».

Таблица 9.3 — Данные, объявляемые при включении режима

Поле	Назначение
Сообщение	Что именно делается и чего ждать пользователю. Показывается крупно, первым
Начало работ	Дата и время начала. Отображается как срок, не как момент включения
Окончание работ	Дата и время окончания. Задаёт срок и одновременно служит предохранителем
Почта и телефон поддержки	Куда обращаться, пока система недоступна. Незаполненное поле не показывается

Начало и окончание задаются целиком: система не примет одну половину окна и не примет окончание раньше начала. По объявленному окну страница считает, сколько времени осталось, и показывает долю прошедшего времени.

Режим снимается сам, когда объявленное окончание прошло. Проверка выполняется при любом обращении к признаку режима, отдельная фоновая задача для этого не нужна. Снятие записывается в журнал событий как `maintenance_auto_disabled`. Поэтому даже забытый включённым режим отпускает пользователей в назначенный срок.

Вход супер-администратора при включённом режиме. Пока режим активен, обычного пользователя уводит на страницу технических работ с

любого адреса, включая форму входа. Для супер-администратора на странице технических работ есть ссылка «Вход для администратора» (адрес `/?admin`) — она открывает форму входа. Обычному работнику эта ссылка не поможет: его учётной записи система откажет во входе, пока идут работы.

Аварийное снятие режима. Если войти в систему не удаётся вовсе — утрачен пароль супер-администратора или недоступен интерфейс, — режим снимается на сервере одной командой. Прикладной сервер перезапускать не нужно: признак режима хранится не дольше десяти секунд.

Листинг 9.2 — Снятие режима технических работ на рабочем сервере

```
make deploy-maintenance-off
```

Ожидаемый результат: команда печатает текущее значение признака (`maintenance.enabled = false`), пользователи возвращаются в систему в течение десяти секунд. Для проверочного стенда команда называется `make staging-maintenance-off`, для локальной установки — `make maintenance-off`.

Восстановление самого доступа супер-администратора выполняется отдельно, порядок описан в подразделе 9.4.

9.6. Переименование базы данных

Если имя базы требуется изменить уже после запуска системы, порядок такой.

Листинг 9.3 — Переименование базы данных

```
make deploy-down
docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
  up -d db
docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
  exec db psql -U postgres \
  -c 'ALTER DATABASE auto_registry RENAME TO buropropuskov;'
```

Затем измените в файле параметров `DB_NAME` и имя базы в конце строки `DATABASE_URL`, после чего запустите систему.

Листинг 9.4 — Запуск после переименования

```
make deploy-up
```

Ожидаемый результат: система запускается, данные на месте. Операция мгновенная, данные не копируются.

ВАЖНО. Переименование выполняется при остановленном прикладном сервере. При активных подключениях база откажется переименовываться.

9.7. Очистка накопленных данных

Часть служебных записей система убирает сама, ежедневно и без участия человека: недействительные маркеры продления сеанса и прочитанные уведомления. Сроки задаются параметрами `REFRESH_TOKEN_RETENTION_DAYS` и `READ_NOTIFICATION_RETENTION_DAYS`, по умолчанию и там, и там 30 суток. Менять их обычно не требуется: уборка касается записей, которыми уже нельзя воспользоваться.

ВАЖНО. Оба параметра не входят в набор, передаваемый через файл параметров. Чтобы изменить срок, добавьте их в раздел `environment` службы `backend` и перезапустите систему, как описано в подразделе 6.1. Значение, прописанное только в `.env`, не подействует.

Всё остальное удаляется только по команде. Разделения на автоматическое и ручное здесь два основания: сама по себе история ценна, а удаление необратимо, и вернуть удалённое можно лишь из резервной копии.

Таблица 9.4 — Группы данных, доступные команде очистки

Группа	Что удаляется	Срок по умолчанию
tokens	Недействительные маркеры продления сеанса	30 суток
notifications	Прочитанные уведомления	30 суток
audit	История действий над сущностями	36 месяцев
snapshots	Суточные слепки таблиц постов	12 месяцев
request-aggregates	Суточные итоги журнала запросов	24 месяца

Команда без указания групп берёт две первые. Значение `all` берёт все пять. Срок задаётся видом `30d` для суток и `12m` для месяцев и заменяет значение по умолчанию для всех выбранных групп.

Ключ `-except` вычитает группы из выбранных. Он избавляет от перечисления: чтобы почистить всё, кроме истории действий, достаточно `-targets=all -except=audit`.

Выборку внутри группы сужают три ключа.

Таблица 9.5 — Ключи сужения выборки

Ключ	Что задаёт	Где применим
<code>-from</code>	Начало периода в виде <code>2023-01-01</code> . Вместе со сроком хранения задаёт отрезок: всё, что старше начала, остаётся нетронутым	Во всех группах
<code>-entity</code>	Тип сущности: <code>car</code> , <code>employee</code> , <code>application</code> и другие	Только в группе <code>audit</code>
<code>-table</code>	Номер таблицы поста	Только в группе <code>snapshots</code>

Ключ, неприменимый к выбранной группе, команда отклоняет и объясняет почему. Это сделано намеренно: если бы такой ключ молча игнорировался, оператор просил бы сузить удаление, а получил бы удаление всей группы. По той же причине проверяется написание типа сущности, а при ошибке выводится перечень допустимых значений.

Без явного разрешения на удаление команда ничего не трогает и только показывает, сколько записей попало бы под условие. Это основной режим: сначала смотрят числа.

В отчёте по каждой группе видно, сколько в ней всего записей, сколько места она занимает и сколько освободит удаление. Оценка освобождаемого объёма приблизительная: записи внутри одной группы разного размера.

Листинг 9.5 — Что будет удалено

```
make deploy-cleanup ARGS="-targets=all"
```

Ожидаемый результат: таблица с числом записей по каждой группе и напоминание, что ничего не удалено.

Листинг 9.6 — Удаление истории старше трёх лет

```
make deploy-cleanup ARGS="-targets=audit -older-than=36m -apply"
```

Ожидаемый результат: та же таблица и строка с числом удалённых записей.

Листинг 9.7 — Очистка всего, кроме истории действий

```
make deploy-cleanup ARGS="-targets=all -except=audit -apply"
```

Ожидаемый результат: в таблице четыре группы вместо пяти, история не затронута.

Листинг 9.8 — История только по транспорту, за отрезок времени

```
make deploy-cleanup ARGS="-targets=audit -entity=car -from=2023-01-01  
-older-than=24m"
```

Ожидаемый результат: под удаление попадают записи о машинах за отрезок от начала 2023 года до границы срока. История людей и заявок не затрагивается, записи старше начала отрезка остаются.

Листинг 9.9 — Слпки одного поста

```
make deploy-cleanup ARGS="-targets=snapshots -table=3 -apply"
```

Ожидаемый результат: удаляются суточные слпки только указанного поста. Номер поста виден в адресной строке при открытии его таблицы.

Справка по всем ключам выводится командой `make deploy-cleanup ARGS="-help"`. На проверочном стенде та же операция называется `make staging-cleanup`.

ВАЖНО. Удаление строк не возвращает место операционной системе. Освободившийся объём остаётся в файлах базы и используется под новые записи, поэтому после очистки размер каталога с данными сразу не уменьшится. Это нормально: место не потеряно, оно просто отдано базе, а не диску.

ВАЖНО. Удаление необратимо. Выполняйте его после создания резервной копии, иначе при ошибке в сроке восстанавливать будет нечего.

ПРИМЕЧАНИЕ. Историю сущностей команда чистит с оговорками. Записи об удалении строк таблиц постов сохраняются независимо от срока: по ним работает корзина. Последние отметки въезда и выезда каждой машины и каждого человека тоже сохраняются: из них берётся время последнего выезда в карточке, и у редко приезжающей машины такая отметка бывает старше любого разумного срока хранения.

ПРИМЕЧАНИЕ. Слепки таблиц, снятые вручную кнопкой в интерфейсе, команда не удаляет никогда. Под срок попадают только суточные, снимаемые системой перед сбросом статусов.

10. Порядок внесения изменений в систему

Раздел описывает полный путь изменения: от получения исходного кода до его появления на рабочем сервере.

10.1. Общая схема

Таблица 10.1 — Этапы внесения изменения

Этап	Где выполняется	Результат
1. Получение исходного кода	Рабочее место разработчика	Локальная копия проекта
2. Внесение правки	Рабочее место разработчика	Изменённый код в отдельной ветке
3. Автоматическая проверка	Конвейер сборки	Отчёт о прохождении проверок
4. Проверка на стенде	Проверочный стенд	Подтверждение работы вживую
5. Выкладывание на рабочий сервер	Рабочий сервер	Изменение доступно пользователям

10.2. Получение исходного кода и внесение правки

Исходный код хранится в системе контроля версий Git. Работа ведётся в отдельной ветке, а не в основной: это позволяет проверить изменение до того, как оно попадёт к другим.

Листинг 10.1 — Получение кода и создание ветки

```
git clone <адрес репозитория> systemburo
cd systemburo
git checkout -b fix/kratkoe-opisanie
```


Локальный запуск для разработки поднимает систему с отдельной базой и не затрагивает ни стенд, ни рабочий сервер.

Листинг 10.2 — Локальный запуск для разработки

```
make up
make seed
```

Ожидаемый результат: интерфейс доступен на порту 8081, программный интерфейс на 8080, вход под учётной записью, созданной командой заполнения.

10.3. Проверка изменения до выкладывания

Перед отправкой изменения прогоняются тесты. Локально достаточно прогнать те, что относятся к изменённой части.

Листинг 10.3 — Прогон проверок

```
make test # бэкенд-тесты
cd frontend && npm run test:run # фронтенд-тесты
cd frontend && npx playwright test # E2E-сценарии
```

Ожидаемый результат: все тесты завершаются успешно. Упавший тест выводит своё имя и расхождение ожидаемого с фактическим.

После отправки изменения конвейер запускает полный набор тестов автоматически, включая сборку образов и сканирование уязвимостей. Изменение, не прошедшее тесты, к выкладыванию не допускается.

10.4. Проверка на стенде

После прохождения тестов и включения изменения в основную ветку оно автоматически выкладывается на проверочный стенд.

На стенде проверяются сценарии, которые автоматика не покрывает: внешний вид, удобство, поведение на реальных данных.

Таблица 10.2 — Проверяемые на стенде сценарии

Сценарий	Что подтверждает
Вход администратора и рядового работника	Проверку подлинности и выдачу прав
Подача заявки со всеми типами вложений	Основной рабочий процесс

Продолжение таблицы 10.2

Сценарий	Что подтверждает
Приём в работу, согласование, завершение	Переходы состояний и запись решений
Открытие таблицы поста, отметка въезда и выезда	Работу постов
Формирование печатной формы по заявке	Работу шаблонов бланков
Выгрузка отчёта	Формирование электронных таблиц
Открытие панели показателей	Расчёт аналитики

10.5. Выкладывание на рабочий сервер

ОПАСНО. Перед выкладыванием создайте резервную копию базы данных. Это единственный способ вернуться назад гарантированно, см. подраздел 10.7.

Порядок действий на рабочем сервере.

1. Предупредить пользователей, при длительном обновлении включить режим технических работ (подраздел 9.5).
2. Получить новую версию исходного кода в каталог системы.

Листинг 10.4 — Обновление исходного кода на рабочем сервере

```
cd /opt/systemburo
git fetch origin
git checkout <версия>
```

1. Пересобрать образы и перезапустить систему.

Листинг 10.5 — Пересборка и перезапуск

```
make deploy-build
make deploy-up
```

1. Проследить за журналом запуска.

Листинг 10.6 — Наблюдение за запуском после обновления

```
make deploy-logs
```

Ожидаемый результат: в журнале нет сообщений уровня `error`. Схема базы приводится в соответствие автоматически именно на этом шаге, и здесь же проявятся несовместимости, если они есть.

1. Выполнить проверки из подраздела 5.9.
2. Снять режим технических работ.

Пользователи не смогут работать с момента перезапуска до успешного старта, обычно это одна-две минуты.

10.6. Изменения схемы базы данных

Схема приводится в соответствие автоматически при каждом запуске прикладного сервера, отдельной команды не требуется. Практические следствия:

- изменения применяются в момент запуска, и если они объёмны, запуск затянется;
- изменения носят добавляющий характер: создаются таблицы, столбцы и индексы, удаления не выполняются;
- при ошибке приведения схемы прикладной сервер завершает работу и не запускается в частично обновлённом состоянии.

10.7. Возврат к предыдущей версии

Возврат кода выполняется получением прежней версии и повторной сборкой.

Листинг 10.7 — Возврат к предыдущей версии кода

```
cd /opt/systemburo
git checkout <предыдущая версия>
make deploy-build
make deploy-up
```

ВАЖНО. Приведение схемы базы выполняется только вперёд. Изменения схемы, внесённые новой версией, при откате кода не отменяются. Поскольку они носят добавляющий характер, прежняя версия обычно продолжает работать. Но если обновление изменило смысл существующих данных, потребуется восстановление базы из копии, созданной до обновления.

11. Резервное копирование и восстановление

11.1. Текущее состояние

Штатная процедура резервного копирования в составе системы отсутствует. Резервное копирование обеспечивается силами организации, где развёрнута система.

ОПАСНО. До настройки резервного копирования отказ дисковой подсистемы или ошибочная операция приведут к безвозвратной утрате всех данных, включая персональные.

Резервное копирование, предоставляемое как услуга хостинга, работает на уровне образа сервера целиком. Оно позволяет вернуть сервер в прежнее состояние, но не заменяет копию базы данных: из образа нельзя извлечь отдельную таблицу, отменить последствия ошибочной операции или перенести данные на другой сервер.

11.2. Что подлежит копированию

Копирования одной базы данных недостаточно.

Таблица 11.1 — Состав данных для резервного копирования

Элемент	Где расположен	Последствия утраты
База данных	Том pgdata	Полная утрата заявок, реестров, журналов, учётных записей
Загруженные файлы	Том uploads	Утрата фотографий, шаблонов бланков, приложений к заявкам. Записи в базе останутся со ссылками на отсутствующие файлы
Файл параметров	.env в каталоге системы	Утрата ключа шифрования персональных данных
Сертификат удостоверяющего центра	nginx/certs/	Потребуется повторный выпуск и повторная установка корневого сертификата на все рабочие места

11.3. Требования к процедуре

- Копия базы создаётся логической выгрузкой средствами СУБД, а не копированием файлов работающей базы: такая копия непригодна для восстановления.

- Ключ шифрования персональных данных хранится отдельно от копий базы. При хранении вместе компрометация архива означает компрометацию персональных данных.
- Копии, содержащие персональные данные, защищаются наравне с самой системой.
- Копии хранятся вне сервера системы: копия на том же диске не защищает от отказа диска.
- Восстановимость проверяется регулярно. Копия, из которой ни разу не восстанавливались, резервной копией не является.

12. Мониторинг и журналы

12.1. Проверка состояния

Адрес `/health` возвращает признак работоспособности прикладного сервера. Проверка подключается к средствам мониторинга организации.

Листинг 12.1 — Проверка состояния

```
curl -s http://localhost/health
```

Ожидаемый результат: `{"status":"ok"}`.

ВАЖНО. Проверка подтверждает, что процесс запущен и отвечает, но к базе данных она не обращается. При недоступной базе проверка продолжит отвечать успешно, а система работать не будет. Полноценным контролем является периодическая проверка входа в систему.

Контейнер прикладного сервера имеет встроенную проверку работоспособности с интервалом 30 секунд. Контейнер базы данных проверяется каждые 10 секунд, и прикладной сервер не запускается, пока база не сообщит о готовности.

12.2. Журналы приложения

Прикладной сервер пишет журнал в поток вывода и, если задан параметр `LOG_FILE_PATH`, дополнительно в файл. На рабочем сервере файловый журнал включён.

Смена файлов выполняется самим приложением, настраивать системное средство ротации не требуется: файл сменяется при достижении 100 МБ, хранится 30 суток, сохраняется до 14 файлов, старые сжимаются.

Листинг 12.2 — Просмотр журналов

```
# все службы, в реальном времени
make deploy-logs

# только прикладной сервер, последние 200 строк
docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
  logs --tail=200 backend

# только ошибки
docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
  logs backend | grep -i error
```

ВАЖНО. Журналы контейнеров по умолчанию не ограничены по размеру и способны заполнить диск. Настройте ограничение при вводе в эксплуатацию.

Листинг 12.3 — Ограничение размера журналов контейнеров

```
# /etc/docker/daemon.json
{
  "log-driver": "json-file",
  "log-opts": { "max-size": "50m", "max-file": "5" }
}
```

После изменения файла выполните перезапуск службы Docker и пересоздание контейнеров.

Листинг 12.4 — Применение ограничения журналов

```
sudo systemctl restart docker
make deploy-up
```

Ожидаемый результат: для каждого контейнера хранится не более пяти файлов журнала по 50 МБ.

12.3. Журналы внутри системы

Часть журналов доступна прямо в интерфейсе и не требует доступа к серверу. Их могут просматривать как специалисты ИТ, так и операторы бюро с соответствующим правом.

Таблица 12.1 — Журналы, доступные в интерфейсе

Журнал	Содержание	Где находится
Журнал запросов	Все обращения к системе: работник, действие, код ответа, длительность, время	Раздел системного управления
Журнал обращений к персональным данным	Обращения к сведениям о людях: работник, действие, раздел, адрес, время	Раздел администрирования, открывается по отдельному праву
Журнал изменений	Кто, когда и что изменил в сущностях системы	Раздел администрирования
Журнал отказов в доступе	Попытки выполнить действие без права	Раздел администрирования
Журнал входов	Входы, выходы, неудачные попытки, блокировки и их снятие администратором	Карточка работника

Журнал запросов и журнал обращений к персональным данным поддерживают фильтрацию и выгрузку в электронную таблицу.

Подробные записи журнала запросов хранятся 30 суток, затем сворачиваются в суточные итоги. Записи журнала обращений к персональным данным хранятся 36 месяцев без свёртки и вместе с журналом запросов не удаляются.

12.4. Что контролировать

Таблица 12.2 — Показатели для наблюдения

Показатель	Норма	Признак неблагополучия
Состояние служб	Все Up, база healthy	Перезапуски, состояние Restarting
Свободное место на диске	Более 20 процентов	Менее 10 процентов требует немедленных действий
Сообщения уровня error в журнале	Единичные	Повторяющиеся однотипные ошибки
Отказы в доступе	Единичные	Массовые отказы указывают на неверно настроенные права
Срок действия сертификата	Более 30 суток	Менее 30 суток требует продления
Число подключений к базе	Значительно меньше 150	Приближение к пределу
Размер базы и крупнейшие таблицы	Рост на 2-4 ГБ в год	Резкий рост одной таблицы вне этого порядка

Размер базы в разрезе таблиц показывает команда `make deploy-storage`. Она только читает и ничего не изменяет, поэтому её можно

выполнять в любое время. Кроме размеров она отвечает, сколько из занятого объёма подлежит очистке по срокам хранения, то есть сразу подсказывает, есть ли смысл браться за подраздел 9.7.

Листинг 12.5 — Обзор занятого места

```
make deploy-storage
```

Ожидаемый результат: размер базы, перечень крупнейших таблиц с числом записей и размером, затем сводка по группам очистки. Число таблиц в перечне задаётся ключом `-top`, например `make deploy-storage ARGS="-top=30"`.

ПРИМЕЧАНИЕ. Разделы журнальных таблиц в перечне не показываются отдельными строками, их объём учтён в родительской таблице. Число записей берётся из внутренней статистики базы и может незначительно расходиться с точным, пока не отработала автоматическая очистка.

13. Диагностика и типовые неисправности

13.1. Порядок диагностики

Таблица 13.1 — Команды диагностики

Что выяснить	Команда
Состояние служб	<code>docker compose -f docker-compose.base.yml -f docker-compose.prod.yml ps</code>
Причина незапуска службы	<code>docker compose -f docker-compose.base.yml -f docker-compose.prod.yml logs <служба></code>
Потребление ресурсов	<code>docker stats</code>
Свободное место	<code>df -h</code> и <code>docker system df</code>
Доступность базы	<code>docker compose -f docker-compose.base.yml -f docker-compose.prod.yml exec db pg_isready -U postgres</code>
Действующие параметры прикладного сервера	<code>docker compose -f docker-compose.base.yml -f docker-compose.prod.yml exec backend env</code>
Проверка сертификата	<code>openssl x509 -in nginx/certs/fullchain.pem -noout -dates</code>

13.2. Система недоступна, страница не открывается

Таблица 13.2 — Диагностика недоступности

Шаг	Действие	Вывод
1	Проверить состояние служб	Службы не запущены, перейти к шагу 2, иначе к шагу 3

Шаг	Действие	Вывод
2	make deploy-up, затем make deploy-logs	Причина незапуска будет в журнале
3	curl -s http://localhost/health	Ответ есть, значит неисправность в сети или правилах межсетевого экрана. Ответа нет, значит в прикладном сервере
4	df -h	Заполнение диска приводит к отказу базы данных

13.3. Обратный прокси не запускается

Наиболее вероятная причина: отсутствует файл сертификата. Прокси с настроенным защищённым соединением не стартует, если сертификата нет.

Листинг 13.1 — Проверка наличия сертификата

```
ls -l nginx/certs/fullchain.pem nginx/certs/privkey.pem
```

Ожидаемый результат: оба файла существуют. При их отсутствии выпустите сертификат по подразделу 7.3.

13.4. Предупреждение о недоверенном сертификате

Признак: браузер сообщает о недоверенном сертификате и не предлагает продолжить.

Отсутствие предложения продолжить нормально и вызвано требованием обязательного защищённого соединения. Причина в том, что корневой сертификат не установлен на рабочем месте, устранение по подразделу 7.4.

Листинг 13.2 — Проверка, какой сертификат отдаёт сервер

```
openssl s_client -connect <адрес>:443 -servername <адрес> \
  </dev/null 2>/dev/null \
  | openssl x509 -noout -subject -ext subjectAltName
```

Ожидаемый результат: в поле альтернативных имён присутствует адрес, по которому обращаются работники. Если обращение идёт по сетевому адресу, а сертификат выпущен только на имя, требуется перевыпуск с указанием адреса вторым аргументом.

13.5. Прикладной сервер не запускается

Таблица 13.3 — Сообщения об ошибках при запуске

Сообщение в журнале	Причина	Устранение
Ошибка обязательного параметра	Не задан или недопустим один из трёх обязательных параметров	Проверить файл параметров, подраздел 6.2
Ошибка подключения к базе	База не готова либо неверны учётные данные	Проверить доступность базы, подраздел 13.1
Ошибка приведения схемы	Несовместимое изменение схемы	Восстановить базу из копии, сделанной до обновления
Требуется ключ шифрования	Включено требование шифрования, ключ отсутствует	Задать ключ либо отключить требование

13.6. Изменение параметра не подействовало

Наиболее частая причина: параметр не входит в набор, передаваемый через файл параметров, см. подраздел 6.1.

Листинг 13.3 — Проверка действующих параметров

```
docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
  exec backend env | grep RESET_TIMEZONE
```

Ожидаемый результат: выведено заданное значение. Если параметр отсутствует в выводе, добавьте его в раздел `environment` файла описания рабочего сервера.

13.7. Заканчивается место на диске

Таблица 13.4 — Освобождение дискового пространства

Потребитель	Как проверить	Как освободить
Журналы контейнеров	<code>docker system df</code>	Настроить ограничение, подраздел 12.2
Неиспользуемые образы	<code>docker system df</code>	<code>docker system prune -a --filter "until=168h"</code>
Журнал запросов в базе	Размер таблиц журнала	Уменьшить <code>REQUEST_LOG_DETAIL_DAYS</code>
История, слепки и суточные итоги	<code>make deploy-storage</code>	Команда очистки, подраздел 9.7
Загруженные файлы	<code>docker system df -v</code>	Перенести том на отдельный раздел

ВАЖНО. Не уменьшайте срок хранения журнала обращений к персональным данным без правовых оснований. Он определяется требованиями к обработке персональных данных.

13.8. Система работает медленно

Таблица 13.5 — Диагностика замедления

Проверка	Толкование
<code>docker stats</code>	Постоянная загрузка процессора базой близко к пределу означает нехватку ресурсов либо тяжёлые отчёты
Журнал запросов, сортировка по длительности	Выявляет конкретные медленные операции
Число подключений к базе	Приближение к 150 указывает на исчерпание предела
Свободная память	Нехватка приводит к вытеснению кэша базы и резкому замедлению

13.9. Работник не видит раздел или получает отказ

1. Открыть журнал отказов в доступе: там зафиксировано, какое именно право не было предоставлено.
2. Проверить назначенные работнику роль и группы прав.
3. Учесть задержку до 30 секунд после изменения прав.
4. Убедиться, что учётная запись не заблокирована.

Отдельный случай - работник не может войти вовсе и видит обратный отсчёт. Это временная блокировка входа после пяти неудачных попыток пароля; в карточке работника она помечена, и там же снимается кнопкой. Журнал входов показывает, сколько попыток и с каких адресов было сделано.

13.10. Не приходят обновления без перезагрузки страницы

Признак: данные на открытой странице обновляются только после ручного обновления.

Проверить, что обратный прокси не буферизует поток событий. В поставляемой конфигурации для адреса потока событий буферизация

отключена и увеличен предельный срок ожидания. Если конфигурация изменялась, восстановите эти параметры.

Принудительное переподключение каждые 10 минут является штатным поведением.

14. Перенос системы на другой сервер

Раздел описывает перенос работающей системы вместе со всеми данными: например, с сервера во внутренней сети организации на арендованный сервер. Порядок обеспечивает сохранность данных и возможность вернуться к прежнему серверу, если что-то пойдёт не так.

14.1. Что переносится

Таблица 14.1 — Состав переносимых данных

Элемент	Где находится	Последствия, если не перенести
База данных	Том pgdata	Утрата всех заявок, реестров, учётных записей и журналов
Загруженные файлы	Том uploads	Фотографии, шаблоны бланков и приложения к заявкам пропадут, записи в базе будут ссылаться в пустоту
Файл параметров	.env в каталоге системы	См. предупреждение ниже
Сертификат	nginx/certs/	Потребуется выпустить заново, что при смене адреса и так необходимо

ОПАСНО. Файл параметров переносится со старого сервера, а не создаётся заново. В нём хранится ключ шифрования персональных данных. Если на новом сервере сгенерировать новый файл параметров, перенесённая база откроется, но сведения документов, удостоверяющих личность, прочитать будет невозможно ничем.

14.2. Что меняется при переносе

Три параметра привязаны к адресу системы и требуют изменения.

Таблица 14.2 — Параметры, зависящие от адреса

Параметр	Что задаёт
CORS_ALLOWED_ORIGINS	Адреса, с которых интерфейсу разрешено обращаться к серверу
VITE_API_BASE_URL	Адрес, по которому интерфейс обращается к серверу

Продолжение таблицы 14.2

Параметр	Что задаёт
COMPOSE_PROJECT_NAME	Имя установки, от которого зависят имена томов с данными

ВАЖНО. Адрес сервера встраивается в интерфейс на этапе сборки, а не читается при запуске. После изменения адреса образ интерфейса необходимо собрать заново, иначе интерфейс на новом сервере продолжит обращаться к старому.

14.3. Подготовка нового сервера

Выполните на новом сервере подразделы 5.1, 5.2 и 5.3: установите служебные пакеты и Docker, распакуйте поставку в рабочий каталог. Создание файла параметров и первый запуск пока не выполняйте.

14.4. Выгрузка данных со старого сервера

Работы выполняются при остановленном прикладном сервере, чтобы данные не менялись во время выгрузки. База данных при этом остаётся запущенной.

Шаг 1. Загрузите параметры установки в оболочку. Без этого имя базы и имя тома окажутся пустыми, и команды выгрузки отработают вхолостую.

Листинг 14.1 — Загрузка параметров в оболочку

```
cd /opt/systemburo
set -a; . ./env; set +a
UPLOADS_VOL=$(docker volume ls --format '{{.Name}}' \
  | grep '_uploads$' | head -1)
echo "база: $DB_NAME, том файлов: $UPLOADS_VOL"
```

Ожидаемый результат: выведены непустые имя базы и имя тома, например база: auto_registry, том файлов: systemburo_uploads. Если что-то пусто, дальше идти нельзя.

Шаг 2. Остановите приём заявок.

Листинг 14.2 — Остановка приёма заявок

```
docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
  stop backend frontend nginx
```

Ожидаемый результат: три службы остановлены, служба базы данных продолжает работать.

Шаг 3. Выгрузите базу данных.

Листинг 14.3 — Выгрузка базы данных

```
docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
  exec -T db pg_dump -U postgres -Fc "$DB_NAME" > /tmp/buro-db.dump
ls -lh /tmp/buro-db.dump
```

Ожидаемый результат: файл создан, размер отличен от нуля и составляет обычно от десятков мегабайт.

Шаг 4. Убедитесь, что выгрузка читается. Повреждённый файл лучше обнаружить сейчас, а не на новом сервере.

Листинг 14.4 — Проверка выгрузки

```
docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
  exec -T db pg_restore --list /dev/stdin < /tmp/buro-db.dump | head -20
```

Ожидаемый результат: выводится перечень объектов базы. Сообщение об ошибке формата означает, что выгрузка повреждена и её нужно повторить.

Шаг 5. Выгрузите загруженные файлы.

Листинг 14.5 — Выгрузка загруженных файлов

```
docker run --rm \
  -v "$UPLOADS_VOL":/data:ro \
  -v /tmp:/backup alpine \
  tar czf /backup/buro-uploads.tar.gz -C /data .
ls -lh /tmp/buro-uploads.tar.gz
```

Ожидаемый результат: архив создан, размер отличен от нуля.

Шаг 6. Передайте на новый сервер три файла: выгрузку базы, архив файлов и файл параметров.

Листинг 14.6 — Передача файлов на новый сервер

```
scp /tmp/buro-db.dump /tmp/buro-uploads.tar.gz .env \
  buro@<адрес нового сервера>:/tmp/
```

ВАЖНО. Файл параметров содержит пароли и ключ шифрования персональных данных. Передавайте его защищённым каналом и удалите промежуточные копии из каталога /tmp на обоих серверах после завершения переноса.

14.5. Восстановление на новом сервере

Шаг 1. Разместите файл параметров в каталоге системы и внесите изменения по подразделу 14.2: новый адрес в `CORS_ALLOWED_ORIGINS` и `VITE_API_BASE_URL`. Строку подключения к базе менять не требуется, она указывает на службу внутри контура.

Листинг 14.7 — Размещение файла параметров

```
cd /opt/systemburo
cp /tmp/.env ./env
chmod 600 ./env
set -a; . ./env; set +a
echo "база: $DB_NAME"
```

Ожидаемый результат: выведено непустое имя базы.

Шаг 2. Запустите базу данных и восстановите выгрузку.

Листинг 14.8 — Запуск базы данных и восстановление

```
docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
  up -d db
sleep 10
docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
  exec -T db pg_restore -U postgres -d "$DB_NAME" --clean --if-exists \
  < /tmp/buro-db.dump
```

Ожидаемый результат: восстановление завершается без сообщений об ошибках. Отдельные предупреждения о несуществующих объектах при первом восстановлении нормальны.

Шаг 3. Проверьте, что данные на месте, ещё до запуска приложения.

Листинг 14.9 — Проверка восстановленной базы

```
docker compose -f docker-compose.base.yml -f docker-compose.prod.yml \
  exec -T db psql -U postgres -d "$DB_NAME" \
  -c "SELECT count(*) AS applications FROM applications;" \
  -c "SELECT count(*) AS users FROM users;"
```

Ожидаемый результат: числа совпадают с прежним сервером. Нули означают, что восстановление не выполнено.

Шаг 4. Восстановите загруженные файлы. Имя тома должно совпадать с тем, которое создаст описание контура.

Листинг 14.10 — Восстановление загруженных файлов

```
UPLOADS_VOL="$(basename $PWD)_uploads"
[ -n "$COMPOSE_PROJECT_NAME" ] \
  && UPLOADS_VOL="${COMPOSE_PROJECT_NAME}_uploads"
docker volume create "$UPLOADS_VOL"
docker run --rm \
  -v "$UPLOADS_VOL":/data \
  -v /tmp:/backup alpine \
  tar xzf /backup/buro-uploads.tar.gz -C /data
docker run --rm -v "$UPLOADS_VOL":/data alpine \
  sh -c "ls /data | head; ls /data | wc -l"
```

Ожидаемый результат: выводится перечень каталогов и их число, совпадающее с прежним сервером.

14.6. Сертификат и запуск

Подготовьте сертификат для нового адреса по разделу 7. При переносе в сеть общего пользования это возможность перейти на сертификат общедоступного удостоверяющего центра и отказаться от установки корневого сертификата на рабочие места.

Листинг 14.11 — Сборка и запуск на новом сервере

```
make deploy-build
make deploy-up
make deploy-logs
```

Ожидаемый результат: службы запускаются, в журнале нет сообщений уровня error. Приведение схемы базы данных выполняется быстро: схема уже соответствует восстановленным данным.

14.7. Проверка после переноса

Проверки выполняются до переключения пользователей на новый сервер.

Таблица 14.3 — Проверки после переноса

Проверка	Ожидаемый результат
Вход администратора	Выполняется с прежним паролем
Число заявок в центре заявок	Совпадает с прежним сервером
Открытие заявки со вложениями	Открывается, файлы и фотографии отображаются
Открытие карточки человека	Сведения документа, удостоверяющего личность, читаются
Открытие таблицы поста	Перечень допущенных объектов на месте

Продолжение таблицы 14.3

Проверка	Ожидаемый результат
Формирование печатной формы	Формируется, шаблон найден
Журналы	Записи за прежний период доступны

ВАЖНО. Проверка чтения сведений документа, удостоверяющего личность, обязательна. Именно она подтверждает, что ключ шифрования перенесён верно. Все остальные проверки могут пройти успешно и при неверном ключе.

14.8. Переключение и вывод прежнего сервера из работы

После успешной проверки измените запись в системе доменных имён либо сообщите работникам новый адрес.

Прежний сервер не выключайте сразу. Держите его остановленным, но сохранным не менее двух недель: этого срока достаточно, чтобы проявились расхождения, незаметные при первичной проверке. Возврат в этом случае сводится к обратному переключению адреса.

Данные, внесённые на новом сервере после переключения, при возврате будут потеряны, поэтому решение о возврате принимается в первые сутки.

По истечении срока хранения удалите с прежнего сервера тома с данными и файл параметров: они содержат персональные данные и ключ шифрования.

Приложение А. Порты и сетевые доступы

Таблица А.1 — Сетевые доступы

Порт	Протокол	Откуда доступен	Назначение
443	TCP	Сеть организации	Работа с системой
80	TCP	Сеть организации	Перенаправление, проверка при выпуске сертификата, проверка состояния
22	TCP	Адреса подразделения ИТ	Управление сервером
8080	TCP	Только внутренняя сеть контейнеров	Прикладной сервер
80	TCP	Только внутренняя сеть контейнеров	Веб-приложение

Продолжение таблицы А.1

Порт	Протокол	Откуда доступен	Назначение
5432	TCP	Только внутренняя сеть контейнеров	База данных

Исходящие соединения требуются только при выпуске сертификата общедоступным центром и при отправке сообщений о сбоях во внешний сервис. Без них система работает полностью автономно.

Приложение Б. Перечень параметров конфигурации

Таблица Б.1 — Подключение и запуск

Параметр	По умолчанию	Назначение
DATABASE_URL	нет, обязателен	Строка подключения к базе данных
DB_NAME	auto_registry	Имя базы данных
DB_USER	postgres	Пользователь базы данных
DB_PASSWORD	нет	Пароль пользователя базы данных
BIND_HOST	0.0.0.0	Адрес приёма соединений
BIND_PORT	8090	Порт прикладного сервера, в поставке задан 8080
LOG_LEVEL	info	Подробность журнала
SWAGGER_ENABLED	false	Публикация описания программного интерфейса

Таблица Б.2 — Маркеры доступа и вход

Параметр	По умолчанию	Назначение
JWT_SECRET	нет, обязателен	Ключ подписи маркера доступа
JWT_REFRESH_SECRET	нет, обязателен	Ключ подписи маркера продления
JWT_ACCESS_TTL	15m	Срок жизни маркера доступа
JWT_REFRESH_TTL	168h	Срок жизни маркера продления
COOKIE_SECURE	true	Передача маркера продления только по защищённому соединению
LOGIN_RATE_LIMIT_MAX	10	Попыток входа с одного адреса за интервал
LOGIN_RATE_LIMIT_WINDOW_SEC	60	Интервал подсчёта попыток, секунды

Таблица Б.3 — Защита персональных данных

Параметр	По умолчанию	Назначение
DATA_ENCRYPTION_KEY	пусто	Ключ шифрования, 64 шестнадцатеричных символа
REQUIRE_ENCRYPTION	false	Запрет запуска без ключа шифрования
PD_AUDIT_RETENTION_MONTHS	36	Срок хранения журнала обращений к персональным данным

Таблица Б.4 — Ограничения и загрузка файлов

Параметр	По умолчанию	Назначение
RATE_LIMIT_PER_MINUTE	200	Запросов в минуту на работника
RATE_LIMIT_WINDOW_SEC	60	Интервал подсчёта
PAGINATION_MAX_LIMIT	100	Наибольший размер страницы выдачи
UPLOAD_PATH	./uploads	Каталог хранения файлов, в поставке /app/uploads
UPLOAD_MAX_FILE_SIZE	10485760	Наибольший размер файла в байтах
UPLOAD_ALLOWED_IMAGE_TYPES	image/jpeg, image/png, image/webp	Разрешённые типы изображений
UPLOAD_ALLOWED_DOC_TYPES	application/pdf и формат документов	Разрешённые типы документов

Таблица Б.5 — Журналы и обслуживание

Параметр	По умолчанию	Назначение
LOG_FILE_PATH	пусто	Файл журнала, пустое значение означает вывод только в поток
LOG_MAX_SIZE_MB	100	Размер файла до смены
LOG_MAX_AGE_DAYS	30	Срок хранения файлов журнала

Продолжение таблицы Б.5

Параметр	По умолчанию	Назначение
LOG_MAX_BACKUPS	14	Число хранимых файлов
LOG_COMPRESS	true	Сжатие при смене
REQUEST_LOG_DETAIL_DAYS	30	Глубина подробных записей журнала запросов
REQUEST_LOG_PARTITION_PRECREATE_DAYS	7	На сколько суток вперёд создаются разделы
REFRESH_TOKEN_RETENTION_DAYS	30	Через сколько суток удаляются недействительные маркеры продления
READ_NOTIFICATION_RETENTION_DAYS	30	Через сколько суток удаляются прочитанные уведомления
ANALYTICS_CACHE_REFRESH_SEC	60	Периодичность пересчёта показателей

Таблица Б.6 — Прочие параметры

Параметр	По умолчанию	Назначение
CORS_ALLOWED_ORIGINS	http://localhost:8081	Адреса, которым разрешены обращения к интерфейсу
RESET_TIMEZONE	Europe/Moscow	Часовой пояс суточного сброса статусов
TELEGRAM_BOT_TOKEN	пусто	Отправка сообщений о сбоях во внешний сервис
TELEGRAM_CHAT_ID	пусто	Адресат сообщений о сбоях
COMPOSE_PROJECT_NAME	имя каталога	Имя установки, определяет имена томов с данными

Приложение В. Файлы, необходимые для запуска

Перечень содержимого поставки, без которого система не соберётся и не запустится.

Таблица В.1 — Обязательные файлы и каталоги

Путь	Назначение
docker-compose.base.yml	Описание состава системы, общее для стенда и рабочего сервера
docker-compose.prod.yml	Дополнение для рабочего сервера: параметры базы, ресурсы, обратный прокси
docker-compose.staging.yml	Дополнение для проверочного стенда
Dockerfile	Сборка образа прикладного сервера
frontend/Dockerfile	Сборка образа веб-приложения
frontend/nginx.conf	Конфигурация отдачи статики внутри образа веб-приложения
nginx/nginx.conf	Конфигурация обратного прокси рабочего сервера
nginx/staging.conf	Конфигурация обратного прокси стенда
nginx/Dockerfile.staging	Сборка образа прокси для стенда: базовый образ дополняется средством создания файла паролей
Makefile	Команды управления
scripts/init-env.sh	Создание файла параметров
scripts/init-tls.sh	Выпуск и продление сертификата
scripts/maintenance-off.sh	Снятие режима технических работ мимо интерфейса
go.mod, go.sum	Перечень библиотек серверной части
cmd/, internal/, docs/	Исходный код серверной части и описание программного интерфейса
frontend/package.json, frontend/package-lock.json	Перечень библиотек интерфейса
frontend/src/, frontend/index.html, frontend/vite.config.js	Исходный код интерфейса

Создаются на сервере при установке и в поставку не входят:

Таблица В.2 — Создаваемое при установке

Путь	Чем создаётся
.env	make init-production
nginx/certs/	scripts/init-tls.sh
nginx/acme-webroot/	scripts/init-tls.sh при выпуске через общедоступный центр

Приложение Г. Проверочный лист ввода в эксплуатацию

Таблица Г.1 — Проверочный лист

№	Проверка	Отметка
1	Операционная система обновлена, часовой пояс задан	
2	Docker и Docker Compose установлены, версии соответствуют требованиям	
3	Дисковый массив собран в зеркало	

Продолжение таблицы Г.1

№	Проверка	Отметка
4	Файл параметров создан, секреты переданы в хранилище организации	
5	Ключ шифрования сохранён отдельно от сервера и от резервных копий	
6	Имя базы данных задано до первого запуска	
7	Сертификат выпущен, срок действия проверен	
8	Корневой сертификат установлен на рабочие места, если применялся собственный центр	
9	Система запущена, все службы работают	
10	Проверка состояния отвечает	
11	Обращение по незащищённому протоколу перенаправляется	
12	Учётная запись администратора создана, пароль отличается от исходного	
13	Вход выполняется, разделы открываются	
14	Правила межсетевого экрана настроены по приложению А	
15	Ограничение размера журналов контейнеров настроено	
16	Резервное копирование настроено, восстановление проверено	
17	Мониторинг подключён	
18	Демонстрационные данные на рабочем сервере отсутствуют	
19	Публикация описания программного интерфейса отключена	
20	Требование шифрования персональных данных включено	

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

В таблице фиксируются изменения документа. Версия повышается при выпуске новой редакции: вторая цифра при уточнениях и дополнениях, первая при переработке структуры или существенном изменении требований.

Таблица - Регистрация изменений

Версия	Дата	Содержание изменения	Внёс
1.0	28.07.2026	Первая редакция	
1.1	28.07.2026	Сверка с реализацией: уточнены количественные показатели, состав прав таблиц постов, поведение поиска, требования к версии Docker Compose, порядок развёртывания стенда и снятия показателей	
1.2	28.07.2026	Описана защита вложения от повторного добавления одного человека или одной машины	
1.3	28.07.2026	Дополнен порядок работы с режимом технических работ: объявляемые сроки и контакты, снятие режима по истечении срока, вход супер-администратора и аварийное снятие командой	
1.4	30.07.2026	Обновлены количественные показатели; описан перенос длинного перечня в бланке вместе с шапкой столбцов	
1.5	30.07.2026	Обновлены количественные показатели: объём кода, число тестов и методов программного интерфейса	
1.6	30.07.2026	Описано обязательное согласие на обработку персональных данных при входе; обновлены количественные показатели	
1.7	30.07.2026	Дополнен состав записи о согласии: редакция текста и его контрольная сумма; описан показ текста согласия на странице документа; обновлены количественные показатели	
1.8	30.07.2026	Обновлены количественные показатели: объём кода, число тестов и сквозных сценариев	
1.9	30.07.2026	Описаны напоминания молчащим согласующим, признак нахождения работника в системе и перечень принимающих заявки	
2.0	30.07.2026	Описан просмотр хода сбора согласий администратором: доля подтвердивших и список остальных	
2.1	30.07.2026	Описана защита от подбора пароля с нарастающим сроком блокировки входа и её досрочное снятие администратором в карточке работника	
2.2	31.07.2026	Описан перенос текста согласия из загруженного документа и вопрос о повторном запросе согласия при сохранении изменённого текста; обновлены количественные показатели	
2.3	31.07.2026	Уточнено, что несуществующий логин запирается на те же сроки и тем же сообщением, что настоящий	

Версия	Дата	Содержание изменения	Внёс
2.4	31.07.2026	Описано скрывание фамилии работника, пока он не дал согласия на обработку персональных данных	
2.5	31.07.2026	Описаны суточная уборка недействительных маркеров продления и прочитанных уведомлений, команда очистки накопленных данных и сроки хранения по группам; обновлены количественные показатели	
2.6	31.07.2026	Уточнено, что сроки автоматической уборки задаются в описании службы, а не в файле параметров	
2.7	31.07.2026	Описаны обзор занятого места по таблицам, исключение групп из очистки и оговорка о том, что удаление строк не возвращает место операционной системе	
2.8	31.07.2026	Описан отзыв согласия на обработку персональных данных работником и администратором, запись выдачи и отзыва в историю учётной записи; уточнён состав сквозного поиска; обновлены количественные показатели	
2.9	31.07.2026	Уточнено, что распознавание опечаток работает при поиске внутри раздела, а сквозной поиск ищет по фрагменту строки и понимает неверную раскладку и написание госномера; обновлены количественные показатели	
3.0	31.07.2026	Описано сужение очистки внутри группы: отрезок времени, тип сущности в журнале действий и отдельный пост для суточных слепков	
3.1	31.07.2026	Уточнено, что до согласия скрываются и рабочие контакты работника, а не только фамилия, имя и отчество; обновлены количественные показатели	